

JD Cortex

Future Work & Extended Work Proposal

Urdu conversational support, system scaling, and repository-level improvements

Tatheer Aima Naqvi · Bushra Ushaq

JD Cortex — Future Work & Extended Work Proposal

Purpose of this document

Part 1 — Urdu Conversational Support

Part 2 — Scaling the System

Part 3 — Repository-Level Improvements

Part 4 — Hardware Sensors for Autonomous Navigation

Suggested Roadmap

Closing note

JD Cortex — Future Work & Extended Work Proposal

Prepared by: Tatheer Aima Naqvi, Bushra Ushaaq **Project:** JD Cortex — Perception, Reasoning, and Memory for an EZ-Robot JD Humanoid

Purpose of this document

This document lays out concrete, scoped extensions to JD Cortex, grouped into three areas: **Urdu language support**, **scaling the system beyond a single robot/single machine**, and **repository-level improvements** identified from reviewing the current codebase. Each item states what exists today, what would change, and roughly how much work it is, so the list can double as a real roadmap rather than a wish list.

Part 1 — Urdu Conversational Support

1.1 Current state

Voice input today goes through `voice_parakeet.py`, which uses a local Parakeet model (via sherpa-onnx) — an English-only speech-to-text engine. Text understanding (`gemini_brain.understand()`) is handled by Gemini, which is already multilingual, but nothing in the pipeline currently captures or transcribes Urdu speech, and JD’s spoken replies go out through ARC’s English TTS voice.

1.2 Proposed additions

Urdu speech-to-text (STT). Replace or supplement the Parakeet model with an Urdu-capable STT engine. Realistic options, roughly in order of ease of integration:

- **Whisper (OpenAI, `large-v3` or `medium`)** — multilingual out of the box, strong Urdu support, runs locally via `faster-whisper` or `whisper.cpp` for reasonable latency on a GPU-equipped machine. This is the most practical first step since it requires no fine-tuning and drops into the existing “record until silence, then transcribe” flow with minimal change to `voice_parakeet.py`’s structure.
- **AI4Bharat IndicConformer / Vakyansh** — models trained specifically for South Asian languages including Urdu, potentially more accurate on local accents and code-switched Urdu-English speech than a general-purpose model.
- **Cloud STT (Google Speech-to-Text, Azure Speech)** — fastest to integrate and generally very accurate for Urdu, at the cost of an internet dependency and per-request billing, which cuts against the project’s current preference for local, offline-capable components.

Language routing. A short language-detection step (or a simple user-facing toggle on the ARC panel) decides which STT engine handles a given recording, since Urdu and English models won’t be equally fast or accurate on the same hardware.

Code-switching. Realistically, spoken commands will mix Urdu and English (“JD, aagay chalo” / “JD, forward chalo”). Whisper handles code-switching reasonably well natively; this should be validated directly against real sample commands before committing to a single model.

Urdu text-to-speech (TTS). ARC’s built-in `SayWait` speaks in whatever voice ARC has configured, which is not Urdu-capable. Two paths: - Route JD’s spoken replies through a separate Urdu TTS engine (e.g. a local Coqui TTS Urdu model, or a cloud TTS API) and play the resulting audio directly rather than through ARC’s voice, when the reply is in Urdu. - Keep ARC’s voice for English and only switch engines for Urdu replies, so the existing English flow is untouched.

Prompting Gemini in Urdu. `gemini_brain.understand()`’s prompt template would need an Urdu variant (or a language flag passed into the existing template) so `REPLY:` comes back in Urdu when the input was Urdu, while `MATCH:` stays in the existing English category/name format the code already parses — action names shouldn’t be translated, since they’re matched literally against `known_actions.py`.

Diary and scene descriptions. `scene_context.py` and `witness_store.diary_context()` currently generate English sentences for the prompt. These could stay in English internally (Gemini reads English fine) even while the spoken conversation is in Urdu — translating the internal prompt scaffolding isn’t necessary, only the user-facing speech.

1.3 Suggested scope for a first pass

A minimal, demonstrable version of this feature doesn’t need every option above. A reasonable first milestone:

1. Add `faster-whisper` as a second STT path alongside Parakeet, selectable by a panel toggle or a config flag.
2. Add an `is_urdu` flag threaded through `understand()`’s prompt so replies come back in Urdu when appropriate.
3. Route Urdu replies through a local Urdu TTS engine instead of ARC’s `SayWait`.
4. Test against ~20 real spoken commands (movement, questions about the diary, casual conversation) covering both pure Urdu and Urdu-English code-switching.

Part 2 — Scaling the System

2.1 Current constraints

The system is built around a single robot, a single ARC instance, and a single machine running all Python processes side by side, coordinated through local files (`scene_state.json` , `speech_queue.json`) and one SQLite database. This is a reasonable design for one robot in one room, but it has natural ceilings:

- `scene_state.json` and `speech_queue.json` are polled on fixed intervals by multiple processes — fine for one robot, but file-based polling doesn’t extend cleanly to multiple robots or multiple machines.
- The witness diary is a single SQLite file in WAL mode — good for one writer (the recorder) and light concurrent reads, but not designed for multiple robots writing to a shared history.
- Every process assumes exactly one ARC instance at `ARC_HOST:ARC_PORT` — there’s no concept of “which robot” anywhere in the code.
- The vision pipeline is tied to one camera feed and runs YOLO detection + pose + face recognition sequentially on whatever hardware is available locally; there’s no separation between “camera intake” and “heavy compute,” so a slow GPU throttles everything downstream.

2.2 Proposed scaling paths

Multi-robot support. Introduce a `robot_id` that threads through scene state, the diary schema, and the speech queue, so one deployment could in principle run several JD units (or other ARC-driven robots) against a shared or per-robot diary. This is a moderate schema change to `witness_store.py` (add a `robot_id` column, index on it) plus updating every reader/writer to filter by it.

Move from file polling to a message bus. Replacing `scene_state.json` / `speech_queue.json` with a lightweight local message broker (Redis pub/sub, or even MQTT for a more IoT-shaped deployment) would remove the polling delay entirely and make it natural to have multiple subscribers (multiple watcher processes, multiple robots) without every consumer re-reading the same file on a timer. This is the single highest-leverage scaling change, since nearly every process in the system currently depends on file polling somewhere.

Separate vision compute from robot control. Right now the vision pipeline needs to run on a machine with both a GPU (for YOLO) and a working ARC connection. Splitting this into a lightweight “frame grabber” (talks to ARC, forwards frames) and a separate, potentially remote “inference worker” (runs YOLO + face recognition, publishes scene state) would let the heavy compute live on a beefier machine or even a small on-prem GPU server, while the robot-facing process stays lightweight and low-latency.

Database migration path. SQLite is a fine choice for a single-robot deployment and shouldn’t be replaced prematurely — but if multi-robot or multi-site diary aggregation becomes a real requirement, a managed Postgres instance (with the same `robot_id`-scoped schema) is a natural next step, since the existing query patterns in `witness_store.py` translate directly.

Containerization. Packaging the vision pipeline, witness recorder, and brain as separate Docker containers (with a shared volume or network for `scene_state.json` during the file-based phase, or shared Redis once migrated) would make the “which dependencies are installed where” problem — currently handled by `run.py check` — mostly disappear, and would make the GPU-dependent vision container easy to run on different hardware than the rest of the stack.

Horizontal scaling of Gemini calls. The existing multi-key rotation in `gemini_brain.py` is really a single-machine quota workaround. At real scale (many robots, many conversations), a small request queue with rate limiting per key, shared across processes, would replace the current per-process key rotation and avoid duplicate quota burn if multiple robots share a Gemini project.

2.3 Suggested scope for a first pass

Scaling to multiple robots is a significant undertaking; a sensible incremental path:

- 1. Add `robot_id` to the diary schema and scene-state file naming, even while still running one robot, so the schema is ready.
- 2. Prototype swapping `speech_queue.json` for Redis pub/sub on a single robot first, to validate the approach before touching multi-robot coordination.
- 3. Only then attempt a second concurrent robot instance against the shared diary.

Part 3 — Repository-Level Improvements

These come directly from reviewing the current code and are worth doing regardless of the Urdu or scaling work, since they reduce risk and maintenance cost.

Area	Observation	Suggested improvement
Testing	No automated test suite exists; only two manual <code>selftest</code> modes (<code>witness_recorder.py</code> , <code>gesture_control.py</code>)	Add <code>pytest</code> coverage for the pure-logic pieces that don’t need hardware — debounce logic, diary trimming, known-action matching, Gemini prompt parsing — using recorded fixture data instead of live camera/ARC input
Duplicate code	<code>shared/known_actions.py</code> and <code>shared/servo_map.py</code> duplicate logic already in <code>jd_robot_system/</code>	Either delete <code>shared/jd_command_and_act.py</code> and its copies if genuinely unused, or explicitly document it as a separate experimental entry point so it doesn’t drift out of sync with the real whitelist
Configuration	Gemini keys and ARC host/port live in <code>config.py</code> with environment-variable overrides; other settings	Consolidate tunable constants into a single <code>.env</code> -driven config module or a YAML file, so tuning the system

	are scattered across per-module constants (<code>ARRIVE_CONFIRM_SECS</code> , <code>STABLE_FRAMES</code> , etc.)	doesn't require editing multiple source files
Panel security	<code>panel_server.py</code> listens on <code>127.0.0.1:5005</code> with a plain-text line protocol and no authentication	Acceptable for a localhost-only deployment; worth adding a shared-secret handshake before any deployment that exposes the port beyond localhost (e.g. remote panel access)
Cross-platform support	ARC and the C# plugin are Windows-only; <code>run.py</code> opens a separate console window per process, which is a Windows-specific pattern	If Linux/macOS support is ever a real goal, <code>run.py</code> 's process-spawning would need a platform-agnostic path (e.g. <code>subprocess</code> with piped output instead of new console windows) — a non-trivial rewrite, best treated as its own project rather than a quick fix
Logging	Processes currently print to their own console window; there's no unified log file across the system	A shared, rotating log file (or per-process logs in one folder) would make debugging a multi-process failure much easier than reading five separate console windows
Privacy/data handling	Enrolled faces, the diary, and snapshots are correctly gitignored, but there's no in-app mechanism to delete a specific person's history on request	Add a small utility (or panel button) to purge all diary entries and encodings for a named person, useful both for privacy and for correcting misidentifications
Model management	YOLO weights download automatically on first run with no version pinning beyond the filename	Pin exact model versions/checksums in <code>requirements.txt</code> or a lockfile, so a fresh setup doesn't silently pick up a newer YOLO release with different behavior
CI	No continuous integration is configured	A basic GitHub Actions workflow running the two existing <code>selftest</code> modes plus any new <code>pytest</code> suite on every push would catch regressions in the pure-logic code without needing camera/ARC hardware in CI

Part 4 — Hardware Sensors for Autonomous Navigation

4.1 Current state

JD currently has no distance-sensing hardware. All spatial awareness comes from the camera-based vision pipeline, which tells the system *what* is visible and roughly *where* in the frame, but not how far away it is or whether the ground ahead is safe to step on. Movement today is limited to pre-approved, fixed Auto Position actions (walk forward, turn, sit) triggered by a command or gesture — JD does not currently decide *how many steps* to take or *stop itself* based on what's physically in front of it.

4.2 Proposed additions

Distance sensors (ultrasonic or ToF). Adding one or more ultrasonic (e.g. HC-SR04) or time-of-flight distance sensors to JD's chassis, wired through the servo controller's spare digital/analog ports, would give the system a real distance-to-obstacle reading instead of relying purely on the camera's 2D

framing. This is the missing piece that would let JD judge “how far is that object” rather than just “is it visible.”

IR sensors for ledge/cliff detection. Downward-facing IR reflectance sensors mounted near JD’s feet (the same principle used in robot vacuums) would detect a sudden drop in the reflected signal when the ground disappears — a step edge, stairs, or a table edge. This is a safety-critical addition: a legged humanoid walking forward on a command has no current way to know the floor is about to end, and a fall is a real risk to the hardware.

Walk-to-object behavior. With distance data available, `main.py` could support a new class of command — “go to the cup,” “come here” — where the vision pipeline identifies the target object’s direction and the distance sensor supplies the exact stopping distance. Rather than a single fixed “walk forward” action, this needs the movement logic to: 1. face the target using the object’s horizontal position in frame, 2. take a bounded number of steps at a time, 3. re-check the distance sensor and camera between each short step (not walk blindly to a calculated distance in one go, since the object or JD’s footing may shift), 4. stop once the distance sensor confirms JD has reached the object.

Automatic ledge stop. Independent of any specific command, the IR ledge sensors should be able to interrupt an in-progress walk immediately — similar in spirit to the existing gesture-control safety stop (which already halts JD if a controlling hand disappears mid-walk). The same pattern applies here: any active forward motion is monitored continuously, and a ledge reading immediately triggers `AutoPositionFrameJump("Standing")` or a dedicated stop action, regardless of what originally triggered the walk (voice command, gesture, or the new walk-to-object behavior).

Sensor fusion, not sensor replacement. None of this replaces the camera pipeline — it complements it. The camera still identifies *what* and roughly *where*; the new sensors supply *how far* and *is the ground safe*, which the camera alone cannot reliably answer.

4.3 Suggested scope for a first pass

- 1. Mount and wire a single forward-facing distance sensor and a pair of downward IR ledge sensors; confirm both can be read through JD’s existing controller board or an added microcontroller bridged back to ARC.
- 2. Add a dedicated safety-stop path that halts any walking action the moment a ledge is detected — this alone is worth doing before any “walk toward an object” logic, since it protects every future movement feature, not just this one.
- 3. Prototype the simplest version of walk-to-object: face the object, take a small fixed number of steps, check distance, repeat — before attempting more precise step-count calculation.

Suggested Roadmap

Term	Focus	Key deliverables
Near-term (next milestone)	Urdu support, first pass	Whisper-based Urdu STT alongside Parakeet, Urdu-aware Gemini prompt, Urdu TTS output, tested against real code-switched commands
Near-term (parallel)	Repository hygiene	pytest coverage for logic-only modules, consolidated config, basic CI
Mid-term	Scaling groundwork	<code>robot_id</code> added to schema, Redis/MQTT prototype replacing file polling on a single robot
Mid-term (parallel)	Vision/compute separation	Split frame-grabbing from inference, so vision compute can move off the

		robot-facing machine
Mid-term (parallel)	Hardware sensors, phase one	Distance sensor + IR ledge sensors mounted and wired, automatic ledge-triggered stop integrated into the movement path
Long-term	Autonomous walk-to-object	Step-by-step approach behavior combining vision-based direction with sensor-based stopping distance
Long-term	True multi-robot deployment	Shared diary across robots, per-robot Gemini quota management, containerized deployment

Closing note

None of the above requires abandoning the current architecture — the project’s core design principle (separate processes for sensing, remembering, deciding, and acting, coordinated through simple shared state) scales naturally in the directions described here. The Urdu work is the most self-contained and demoable extension; the scaling work is the most structurally significant; and the hardware sensor work is the most safety-critical, since it’s the first feature that would let JD move under its own judgment rather than only on a direct command. It should be sequenced with the ledge-detection safety stop built and proven first, before any autonomous walk-to-object behavior is attempted.